

AcadEval+

Complete Tech-Stack-Aligned Implementation Plan

From Training Datasets to a Deployed, Module-by-Module Build

Frontend	React.js
Backend	FastAPI (Python)
Database	PostgreSQL
AI / NLP	Hugging Face Transformers (BERT, SBERT), spaCy
Machine Learning	Scikit-learn
LLM Integration	Google Gemini API / Ollama
Document Processing	PyMuPDF, python-docx, python-pptx
Citation & Reference Analysis	GROBID, AnyStyle
Writing Quality Analysis	textstat, TextBlob
Explainable AI	LIME / SHAP
Task Processing	Celery + Redis
Authentication	JWT
External APIs	Semantic Scholar API, GitHub API, Moodle REST API
Email Service	Gmail SMTP
Deployment	Docker, Render, Vercel
Version Control	Git & GitHub

Note: iThenticate (plagiarism-check) integration is intentionally deferred — planned for a later phase, not implemented in this plan.

Contents

1. How This Plan Maps to the Tech Stack
2. Datasets — What to Build/Collect Before Any Module Code
3. System Architecture
4. Implementation Pipeline Overview
5. Module 1 — Document Ingestion & Parsing
6. Module 2 — Domain & Sub-domain Classification
7. Module 3 — Entity / Feature Extraction
8. Module 4 — Project Knowledge Graph Construction
9. Module 5 — Graph-Based Novelty Engine
10. Module 6 — Citation & Reference Analysis
11. Module 7 — Writing Quality Analysis
12. Module 8 — Research Trend Scoring
13. Module 9 — Explainable AI Layer
14. Module 10 — Faculty Review & Feedback Loop
15. Module 11 — Asynchronous Task Processing
16. Module 12 — Authentication & Access Control
17. Module 13 — External Integrations
18. Module 14 — Notification Service
19. Module 15 — Web Platform (Frontend + Backend)
20. Module 16 — Deployment & DevOps
21. Deferred: iThenticate Integration
22. Build Order & Milestones

1 How This Plan Maps to the Tech Stack

Every module below is built strictly from the tools already chosen — nothing introduces a new paid service or an extra moving part unless explicitly noted (the one exception is NetworkX, a free Python graph library needed to run graph algorithms since no dedicated graph database was in the original list — it runs inside the existing FastAPI/PostgreSQL backend, no new service to deploy).

Tech Stack Item	Module(s) That Use It
React.js	Module 15 — Web Platform (all screens)
FastAPI (Python)	Module 15 — every module is exposed as a FastAPI endpoint
PostgreSQL	Module 4 (graph nodes/edges), Module 10 (faculty scores), all dataset storage
HuggingFace Transformers (BERT, SBERT)	Module 2 (domain embeddings), Module 3 (entity verification), Module 5 (project embeddings)
spaCy	Module 3 — entity/feature extraction (NER + EntityRuler)
Scikit-learn	Module 5 — novelty signal combination (regression), Module 9 (model backing SHAP)
Google Gemini API / Ollama	Module 2 (tie-breaking), Module 3 (entity verification)
PyMuPDF, python-docx, python-pptx	Module 1 — document ingestion & parsing
GROBID, AnyStyle	Module 6 — citation & reference analysis
textstat, TextBlob	Module 7 — writing quality analysis
LIME / SHAP	Module 9 — explainable AI layer
Celery + Redis	Module 11 — async processing for every slow module (1, 2, 3, 4, 5, 6)
JWT	Module 12 — authentication for students/faculty
Semantic Scholar API, GitHub API, Moodle REST API	Module 13 — trend data, public-repo checks, LMS auto-import
Gmail SMTP	Module 14 — notifications
Docker, Render, Vercel	Module 16 — deployment
Git & GitHub	Version control across every module

2 Datasets — What to Build/Collect Before Any Module Code

Every module below depends on at least one of these datasets existing first. Building code before the data it needs just means rewriting the code later — build these in this order.

#	Dataset	What It Contains	Feeds
1	AcadEval_Corpus	Raw historical project proposals (title, abstract, tech stack, faculty notes if available)	Everything
2	AcadEval_DomainTaxonomy	~100-300 domain/sub-domain entries (ACM/IEEE classification + GitHub topics + manual curation)	Module 2
3	AcadEval_FeatureKnowledgeBase	~300-500 curated entities: algorithms, technologies, datasets, applications, with aliases	Module 3
4	AcadEval_ProjectGraphBank	Every project's graph, stored as PostgreSQL node/edge rows	Modules 4, 5
5	AcadEval_CitationBenchmark	A small set of proposals with faculty-reviewed citation quality notes (enough real references? credible sources?)	Module 6
6	AcadEval_WritingQualityBenchmark	A small faculty-scored sample for writing clarity, used only to sanity-check textstat/TextBlob thresholds	Module 7
7	AcadEval_TrendBase	Topic / year / paper count / citation count pulled from Semantic Scholar	Module 8
8	AcadEval_FacultyEvaluation	Faculty novelty ratings + comments — the ground truth for validation	Modules 5, 9, 10
9	AcadEval_NovelBenchmark	Final fused dataset: graph signals + citation score + writing score + trend + faculty label	Research validation

Why datasets 5 and 6 are deliberately small: citation-quality and writing-quality checks lean mostly on off-the-shelf libraries (GROBID/AnyStyle, textstat/TextBlob) that need calibration, not full training — a faculty-reviewed sample of 30-50 proposals is enough to pick sensible thresholds and confirm the metrics track real quality.

3 System Architecture

AcadEval+ System Architecture

(aligned to the chosen tech stack)

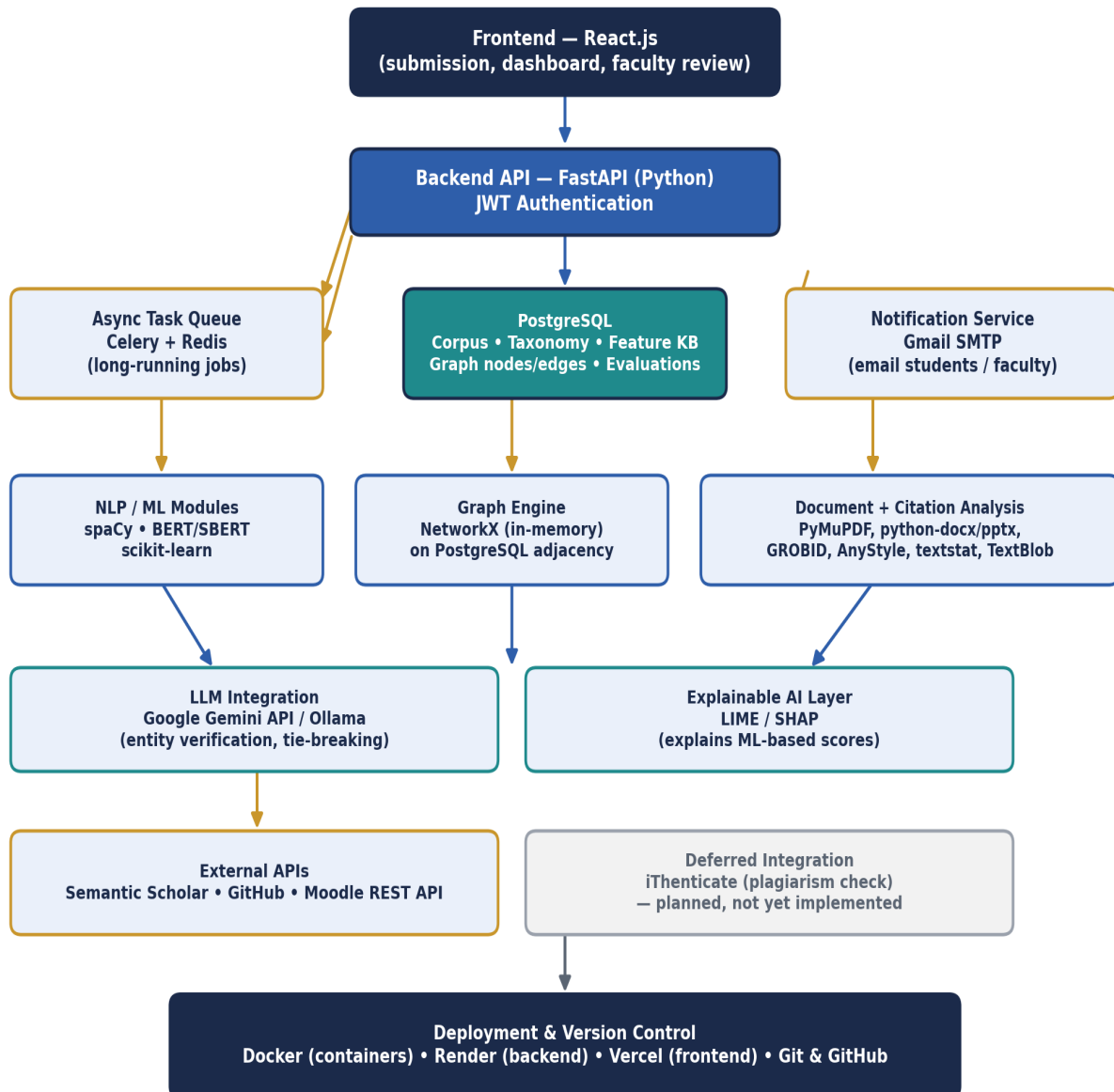


Figure 1. Every box maps directly to a tech-stack item — no component here is unaccounted for in Section 1.

4 Implementation Pipeline Overview

AcadEval+ Implementation Pipeline

(module order, mapped to the chosen tech stack)

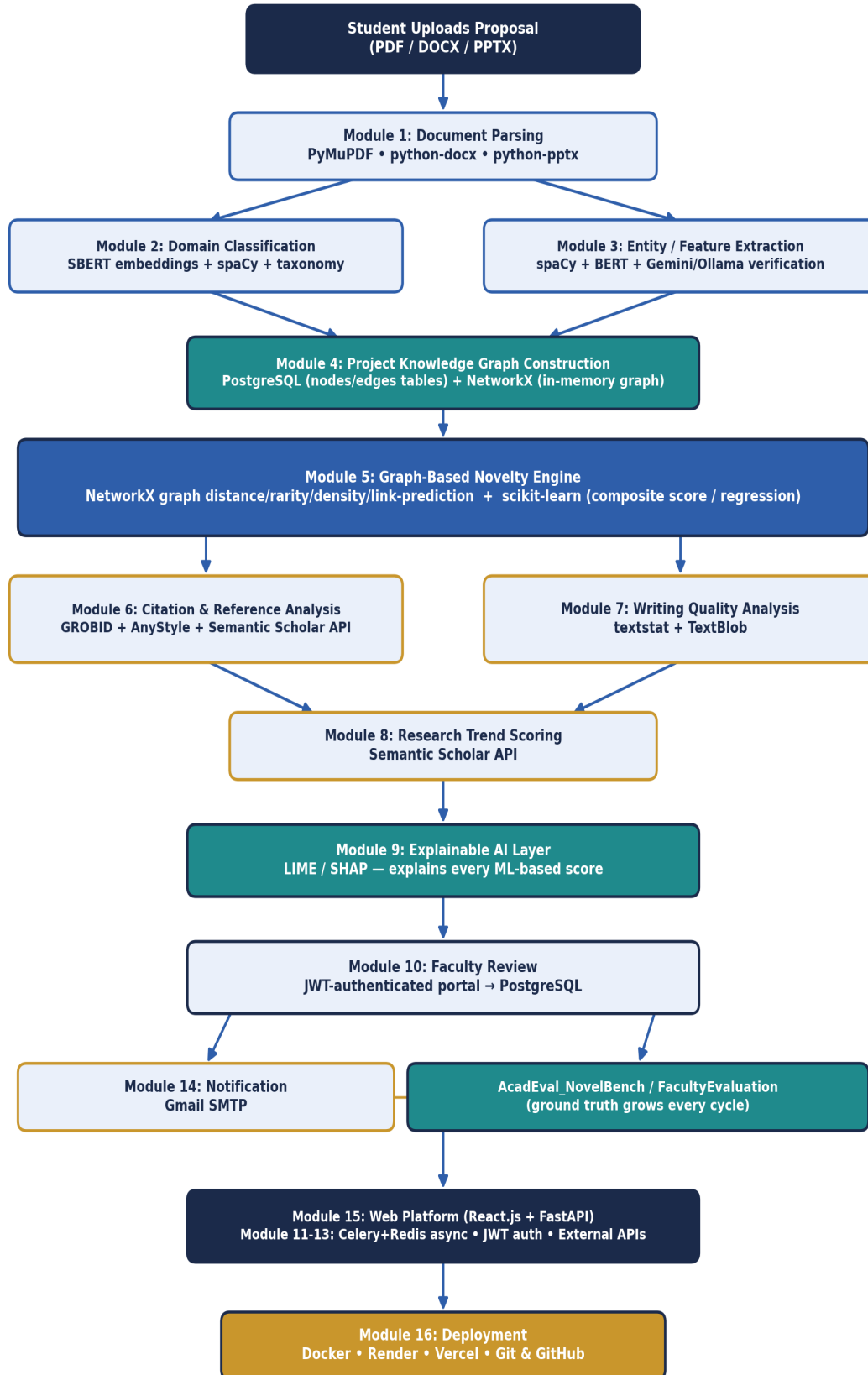


Figure 2. Full module order, from upload to deployment.

5 Module 1 — Document Ingestion and Parsing

What it is: the entry point of the whole system — takes an uploaded proposal file (PDF, Word, or PowerPoint) and turns it into clean, structured text: title, abstract, sections, and any reference list, ready for every downstream module.

Why it matters: every other module assumes clean text is already available. If parsing is sloppy here, every score downstream inherits the error.

Step-by-step implementation

- 1 Accept file upload in FastAPI (POST /submit) as multipart/form-data; store the raw file in a per-project folder (or S3-compatible bucket later).
- 2 Detect file type by extension/MIME type and route to the matching parser: PyMuPDF (fitz) for PDF, python-docx for .docx, python-pptx for .pptx.
- 3 Extract raw text, plus structural hints where possible (headings, page numbers) — PyMuPDF can pull font-size metadata, which helps guess where the abstract/title sit.
- 4 Run a lightweight heuristic (or a short Gemini/Ollama prompt) to split the raw text into: title, abstract, body sections, and reference list.
- 5 Store the parsed result as a JSON blob attached to the project row in PostgreSQL, and hand it off (via a Celery task — Module 11) to Module 2 and Module 3.

Reference implementation

```
import fitz # PyMuPDF
from docx import Document
from pptx import Presentation

def parse_pdf(path):
    doc = fitz.open(path)
    return "\n".join(page.get_text() for page in doc)

def parse_docx(path):
    doc = Document(path)
    return "\n".join(p.text for p in doc.paragraphs)

def parse_pptx(path):
    prs = Presentation(path)
    slides_text = []
    for slide in prs.slides:
        for shape in slide.shapes:
            if shape.has_text_frame:
                slides_text.append(shape.text_frame.text)
    return "\n".join(slides_text)
```

6 Module 2 — Domain and Sub-domain Classification

What it is: assigns every project to a domain and sub-domain (e.g. Artificial Intelligence -> Computer Vision), using AcadEval_DomainTaxonomy.

Why it matters: without domain scoping, the system would compare an ECG project against a blockchain project — a meaningless comparison.

Step-by-step implementation

- 1 Build the taxonomy file (~100-300 leaf entries) from ACM Computing Classification, IEEE taxonomies, GitHub Topics, and manual curation; store it in PostgreSQL as a domain/sub-domain table with a parent_id column.
- 2 Load an SBERT model (Sentence-Transformers, e.g. all-MiniLM-L6-v2) once at startup; pre-compute and cache an embedding for every taxonomy leaf's name + keywords.
- 3 For every incoming project (post Module 1), embed the parsed abstract with the same SBERT model.
- 4 Score every taxonomy leaf by cosine similarity to the project embedding; keep the top match.
- 5 If the top-2 scores are within 0.05 of each other, send both candidates to a short Gemini/Ollama tie-breaker prompt instead of guessing.
- 6 Write the winning domain/sub-domain onto the project's row in PostgreSQL.
- 7 Validate on ~50 faculty-reviewed projects (top-1 accuracy) before trusting the classifier downstream.

Reference implementation

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
taxonomy_embeddings = model.encode(taxonomy_leaf_texts) # cached once

def classify_domain(abstract: str):
    query_emb = model.encode(abstract)
    scores = util.cos_sim(query_emb, taxonomy_embeddings)[0]
    top2 = scores.topk(2)
    if (top2.values[0] - top2.values[1]) < 0.05:
        return llm_tiebreak(abstract, candidates=top2.indices)
    return taxonomy_leaves[top2.indices[0]]
```

7 Module 3 — Entity and Feature Extraction

What it is: pulls structured entities — algorithms, datasets, technologies, application, metrics — out of the parsed text, using AcadEval_FeatureKnowledgeBase as the vocabulary.

Why it matters: this turns unstructured text into the building blocks the project graph (Module 4) is made of.

Step-by-step implementation

- 1 Build AcadEval_FeatureKnowledgeBase (~300-500 curated entries: name, category, aliases, first_seen_year) in PostgreSQL.
- 2 Auto-generate a spaCy EntityRuler pattern list directly from the FeatureKnowledgeBase name/alias columns.
- 3 Run the parsed abstract through spaCy: the base model catches generic entities, the EntityRuler catches every known FeatureKnowledgeBase term.
- 4 For spans spaCy could not match, run a BERT-based similarity check first (embed the span and compare to FeatureKnowledgeBase embeddings) as a cheap first pass.
- 5 If still unmatched, send the span to a Gemini/Ollama verification prompt — map it to the closest entry, or confirm it as a new feature candidate.
- 6 Route confirmed new-feature candidates to a manual review queue; once approved, add them to FeatureKnowledgeBase.
- 7 Emit a structured JSON per project: domain, subdomain, algorithms[], technologies[], datasets[], application, metrics[] — this feeds Module 4 directly.

EntityRuler pattern example

```
import spacy
nlp = spacy.load("en_core_web_sm")
ruler = nlp.add_pipe("entity_ruler", before="ner")

patterns = [
    {"label": "ALGORITHM", "pattern": "FaceNet"},
    {"label": "TECHNOLOGY", "pattern": "Edge Computing"},
    {"label": "DATASET", "pattern": "LFW"},
] # generated automatically from FeatureKnowledgeBase
ruler.add_patterns(patterns)
```

8 Module 4 — Project Knowledge Graph Construction

What it is: converts each project's structured entity list (Module 3 output) into a small graph — the project as a central node connected to its domain, algorithms, technologies, datasets, and application — stored relationally in PostgreSQL and operated on in memory with **NetworkX** (the tech-stack list does not include a graph database, so the graph itself lives as ordinary rows and is loaded into a NetworkX graph object whenever it needs to be analysed).

Why it matters: this is the core representation shift that makes novelty explainable — projects become structured, comparable knowledge instead of paragraphs of text.

Data model (PostgreSQL)

```
CREATE TABLE graph_nodes (
  id SERIAL PRIMARY KEY,
  node_type VARCHAR NOT NULL, -- 'Project', 'Domain', 'Algorithm', 'Technology', 'Dataset', 'Application'
  name VARCHAR NOT NULL,
  UNIQUE(node_type, name)
);

CREATE TABLE graph_edges (
  id SERIAL PRIMARY KEY,
  from_node INT REFERENCES graph_nodes(id),
  to_node INT REFERENCES graph_nodes(id),
  relationship VARCHAR NOT NULL, -- 'HAS_DOMAIN', 'USES_ALGORITHM', etc.
  confidence NUMERIC
);
```

Step-by-step implementation

- 1 Create the `graph_nodes` / `graph_edges` tables above, with a unique constraint on `(node_type, name)` so shared entities (e.g. “CNN” used by 50 projects) are stored once, not duplicated.
- 2 For every incoming project, upsert its Project node, then upsert one node per entity from Module 3's JSON, then insert the connecting edges — all inside a single database transaction.
- 3 Write a small helper, `load_graph()`, that reads all rows from `graph_nodes/graph_edges` and builds a NetworkX MultiDiGraph in memory — this is what Module 5 actually analyses.
- 4 Cache the loaded NetworkX graph (e.g. in Redis, refreshed after every new ingestion) so Module 5 does not rebuild it from scratch on every request.

Reference implementation — ingestion and graph loading

```

def upsert_node(cur, node_type, name):
    cur.execute("""INSERT INTO graph_nodes (node_type, name) VALUES (%s, %s)
                  ON CONFLICT (node_type, name) DO UPDATE SET name = EXCLUDED.name
                  RETURNING id""", (node_type, name))
    return cur.fetchone()[0]

def ingest_project(cur, project_json):
    proj_id = upsert_node(cur, "Project", project_json["project_id"])
    dom_id = upsert_node(cur, "Domain", project_json["domain"])
    cur.execute("INSERT INTO graph_edges (from_node, to_node, relationship) VALUES (%s,%s,'HAS_DO
MAIN')",
                (proj_id, dom_id))
    for algo in project_json["algorithms"]:
        a_id = upsert_node(cur, "Algorithm", algo["name"])
        cur.execute("INSERT INTO graph_edges (from_node, to_node, relationship, confidence)"
                    " VALUES (%s,%s,'USES_ALGORITHM',%s)", (proj_id, a_id, algo["confidence"]))
    # ... same pattern for technologies, datasets, application

import networkx as nx
def load_graph(cur):
    G = nx.MultiDiGraph()
    cur.execute("SELECT id, node_type, name FROM graph_nodes")
    for id_, ntype, name in cur.fetchall():
        G.add_node(id_, type=ntype, name=name)
    cur.execute("SELECT from_node, to_node, relationship FROM graph_edges")
    for f, t, rel in cur.fetchall():
        G.add_edge(f, t, relationship=rel)
    return G

```

9 Module 5 — Graph-Based Novelty Engine

What it is: computes the actual novelty score from the graph, using five explainable signals, computed with NetworkX and combined with scikit-learn.

Why it matters: this is the core research contribution — novelty comes from graph structure, not a manually assigned weighting formula.

Signal 1 — Graph Distance

Generate a structural embedding for every Project node (Node2Vec, via the node2vec PyPI package which runs directly on a NetworkX graph), then compare the new project's vector to every existing one with scikit-learn's cosine_similarity.

```
from node2vec import Node2Vec
from sklearn.metrics.pairwise import cosine_similarity

n2v = Node2Vec(G, dimensions=64, walk_length=15, num_walks=100)
model = n2v.fit(window=5, min_count=1)
vec = model.wv[str(new_project_node_id)]
sims = cosine_similarity([vec], existing_project_vectors)[0]
graph_distance = 1 - sims.max()
```

Signal 2 — Feature Rarity

For every entity node the new project connects to, rarity is the inverse of its in-degree (how many other Project nodes already connect to it):

```
def rarity(G, entity_id):
    degree = sum(1 for _, _, d in G.in_edges(entity_id, data=True)
                 if G.nodes[_]['type'] == 'Project')
    return 1 / (degree + 1)
```

Signal 3 — Relationship Rarity

Counts how many projects share the exact same entity pair (e.g. Algorithm -> Application) via a direct SQL aggregate query — faster than a graph traversal for this specific check:

```
SELECT COUNT(DISTINCT e1.from_node) AS pair_count
FROM graph_edges e1
JOIN graph_edges e2 ON e1.from_node = e2.from_node
WHERE e1.to_node = :algorithm_id AND e2.to_node = :application_id;
```

Signal 4 — Graph Density

Uses NetworkX's local clustering coefficient around the project's domain neighbourhood: a crowded, tightly-interconnected neighbourhood scores high (low novelty); a sparse one scores low (high novelty).

```
import networkx as nx
undirected = G.to_undirected()
density = nx.clustering(undirected, nodes=[new_project_node_id])[new_project_node_id]
```

Signal 5 — New-Connection Discovery

Checks whether the specific entity pair this project introduces has ever co-occurred, using NetworkX's Adamic-Adar link-prediction score between the two entity nodes:

```
preds = list(nx.adamic_adar_index(undirected, [(algorithm_id, application_id)]))
aa_score = preds[0][2]
# aa_score == 0 combined with pair_count == 0 (Signal 3) => never proposed before
```

Combining the five signals

Not a fixed percentage formula. Two honest, defensible options:

- **v1 — documented composite index:** an unweighted average of the five normalized signals, used until enough faculty-labeled data exists.
- **v2 — learned model:** once AcadEval_FacultyEvaluation has enough rows, train a scikit-learn Ridge regression (five signals as input features, faculty score as target); its coefficients become evidence for which signals matter most, and Module 9 (SHAP) explains individual predictions.

10 Module 6 — Citation and Reference Analysis

What it is: checks the quality of a proposal's references — are there enough citations, are they real and traceable, and are they recent and relevant — using GROBID and AnyStyle.

Why it matters: citation quality is a separate signal from novelty; a highly novel idea backed by weak or fabricated references is still a red flag a guide needs to see.

Step-by-step implementation

- 1 Run the uploaded PDF through a GROBID server (self-hosted via Docker — grobid/grobid image) to extract a structured reference list (authors, title, year, venue) from the document's bibliography section.
- 2 For proposals submitted as plain text/Word (no clean bibliography section), fall back to AnyStyle to parse loosely-formatted citation strings into structured fields.
- 3 Cross-check each parsed reference against the Semantic Scholar API (Module 8's API client) to confirm it corresponds to a real paper, and pull its publication year and citation count.
- 4 Compute simple, explainable sub-scores: `reference_count`, `percent_verified` (found on Semantic Scholar), `average_reference_age`, `percent_recent` (published in the last 5 years).
- 5 Store these sub-scores on the project record; surface them in the Module 9 explainable report alongside the novelty signals — kept as a separate section, not blended into the Novelty Index.

Reference implementation — GROBID call

```
import requests

def extract_references(pdf_path):
    files = {"input": open(pdf_path, "rb")}
    resp = requests.post("http://localhost:8070/api/processReferences", files=files)
    return parse_tei_xml(resp.text) # GROBID returns TEI XML; extract author/title/year
```


11 Module 7 — Writing Quality Analysis

What it is: scores the readability and basic writing quality of the abstract/proposal text using textstat and TextBlob.

Why it matters: this is a supporting quality signal for the guide's overall report — independent of novelty — flagging proposals that may need editing before submission.

Step-by-step implementation

- 1 Run textstat on the parsed abstract/body text to compute standard readability scores (Flesch Reading Ease, Gunning Fog).
- 2 Run TextBlob on the same text for sentence-level polarity/subjectivity (flagging overly informal or opinion-heavy writing) and basic spelling-error counts.
- 3 Compare the scores against the small AcadEval_WritingQualityBenchmark (Section 2) to pick sensible cut-off bands (e.g. “clear”, “needs editing”) rather than inventing arbitrary thresholds.
- 4 Attach the writing-quality band and its underlying numbers to the project's explainable report (Module 9).

Reference implementation

```
import textstat
from textblob import TextBlob

def writing_quality(text):
    return {
        "flesch_reading_ease": textstat.flesch_reading_ease(text),
        "gunning_fog": textstat.gunning_fog(text),
        "subjectivity": TextBlob(text).sentiment.subjectivity,
    }
```

12 Module 8 — Research Trend Scoring

What it is: adds a real-world signal — is this topic currently rising or declining in the research literature — using AcadEval_TrendBase, built from the Semantic Scholar API.

Step-by-step implementation

- 1 Call the Semantic Scholar Graph API for each taxonomy leaf topic and/or extracted algorithm/technology name.
- 2 Store yearly paper counts and citation counts in AcadEval_TrendBase (topic, year, paper_count, citation_count) in PostgreSQL.
- 3 Compute a trend score as the year-over-year growth rate of paper count over the last 3-5 years.
- 4 Attach the trend score to the project record so Module 9's report can show it as context next to the novelty signals.

```
GET https://api.semanticscholar.org/graph/v1/paper/search?query=vision+transformer+agriculture&fields=year,citationCount&limit=100
```

```
trend_score = (papers_last_year - papers_3_years_ago) / max(papers_3_years_ago, 1)
```

13 Module 9 — Explainable AI Layer

What it is: explains every ML-based score in plain terms, using LIME and SHAP, so a guide can see **why** a project got the score it did, not just the number.

Why it matters: reviewers and faculty will not trust a black-box number; explainability is also a research requirement for the eventual paper.

Step-by-step implementation

- 1 For the scikit-learn regression model that combines the five novelty signals (Module 5, v2), attach SHAP's LinearExplainer (or TreeExplainer if a gradient-boosted model is used instead) to get per-project, per-signal contribution values.
- 2 For the domain classifier (Module 2, an embedding nearest-neighbour lookup rather than a trained model), use LIME's text explainer on a simple bag-of-words proxy classifier trained on the same taxonomy labels, purely to highlight which words in the abstract pushed the classification — this is a supporting/illustrative explanation, not the classifier itself.
- 3 Render both explanations as a small bar chart (signal or word -> contribution) in the web dashboard, next to the plain-language explanation lines from Module 5's report.

Reference implementation — SHAP on the novelty regression model

```
import shap

explainer = shap.LinearExplainer(novelty_model, background_data)
shap_values = explainer.shap_values(new_project_signal_vector)
# shap_values[i] = how much signal i pushed the score up or down for this project
```

14 Module 10 — Faculty Review and Feedback Loop

What it is: lets faculty confirm or override the system's score, behind JWT-authenticated login.

Why it matters: this produces AcadEval_FacultyEvaluation, the ground truth that validates — or disproves — the whole method.

Data model (PostgreSQL)

```
CREATE TABLE faculty_evaluation (  
  id SERIAL PRIMARY KEY,  
  project_id VARCHAR REFERENCES projects(id),  
  faculty_id VARCHAR NOT NULL,  
  system_score NUMERIC,  
  faculty_score NUMERIC NOT NULL,  
  comment TEXT,  
  reviewed_at TIMESTAMP DEFAULT now()  
);
```

Step-by-step implementation

- 1 Build a review screen (React) showing the Module 9 report, with a numeric novelty rating field (1-10) plus an optional comment, reachable only after JWT login (Module 12) with a faculty role.
- 2 On submit, insert a row into faculty_evaluation via POST /faculty-review.
- 3 Run a scheduled job (a Celery beat task, Module 11) that recomputes the correlation between system_score and faculty_score as the table grows — this becomes a core evaluation result for the eventual paper.
- 4 Feed confirmed/overridden scores back to enrich AcadEval_NovelBench.

```
from scipy.stats import spearmanr  
corr, p_value = spearmanr(df['system_score'], df['faculty_score'])
```

15 Module 11 — Asynchronous Task Processing

What it is: runs every slow operation (document parsing, embedding, GROBID calls, LLM calls, graph analysis) in the background using Celery and Redis, so the web app stays responsive.

Why it matters: without this, a student submitting a proposal would sit on a spinning page for the 10-30 seconds these steps can realistically take.

Step-by-step implementation

- 1 Run a Redis instance (Docker container) as the Celery broker and result backend.
- 2 Wrap Modules 1, 2, 3, 4, 5, and 6 each as a Celery task; chain them with Celery's `chain()` primitive so a submission triggers the whole pipeline in order.
- 3 The FastAPI POST `/submit` endpoint enqueues the chain and immediately returns a `job_id`; the frontend polls GET `/status/{job_id}` (or uses a WebSocket) until the report is ready.
- 4 Add a Celery beat schedule for periodic jobs: refreshing `AcadEval_TrendBase` weekly, recomputing the faculty-score correlation nightly.

```
from celery import chain

@celery_app.task
def parse_task(project_id): ...

@celery_app.task
def classify_task(project_id): ...

@celery_app.task
def extract_entities_task(project_id): ...

pipeline = chain(parse_task.s(project_id), classify_task.s(), extract_entities_task.s())
pipeline.apply_async()
```

16 Module 12 — Authentication and Access Control

What it is: login and role-based access (student vs. faculty) using JWT.

Step-by-step implementation

- 1 Store users (id, email, password_hash, role) in PostgreSQL; hash passwords with bcrypt/passlib.
- 2 On login (POST /auth/login), verify the password and issue a signed JWT containing user_id and role, with a short expiry plus a refresh token.
- 3 Protect every FastAPI route with a dependency that decodes and validates the JWT, rejecting expired/invalid tokens and enforcing role checks (e.g. /faculty-review requires role=faculty).
- 4 Store the JWT in an HTTP-only cookie or in memory on the React frontend (not localStorage, to reduce XSS exposure) and attach it to every API call.

```
from fastapi import Depends, HTTPException
from jose import jwt

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return payload # {user_id, role, exp}
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

17 Module 13 — External Integrations

What it is: the three outside data sources the platform talks to: Semantic Scholar (trend data, Module 8), GitHub (public-repository lookups), and Moodle REST API (auto-importing student submissions from the college LMS).

Step-by-step implementation

- 1 Semantic Scholar: a single thin client wrapping the paper/search endpoint, with local caching in PostgreSQL (AcadEval_TrendBase) to avoid re-querying the same topic repeatedly.
- 2 GitHub API: given an extracted technology/algorithm name, search public repositories mentioning the same combination — an optional, supporting signal that a similar open-source implementation already exists (surfaced in the report, not folded into the Novelty Index).
- 3 Moodle REST API: a scheduled Celery task that pulls newly submitted assignment files from a configured Moodle course via its web service API, and feeds them into Module 1 automatically instead of requiring manual upload.

```
# Moodle REST API example
GET {moodle_url}/webservice/rest/server.php
    ?wstoken={token}
    &wsfunction=mod_assign_get_submissions
    &assignid={assignment_id}
    &moodlewsrestformat=json
```

18 Module 14 — Notification Service

What it is: emails students and faculty at the right moments — report ready, faculty review pending — using Gmail SMTP.

Step-by-step implementation

- 1 Create a dedicated Gmail account for the platform and generate an app password for SMTP access (not the main account password).
- 2 Wrap `smtplib` in a small `send_email(to, subject, body)` helper, called as a Celery task (Module 11) at the end of the report-generation chain and after a faculty review is submitted.
- 3 Keep templates simple and plain-text/HTML hybrid so they render acceptably in any mail client.

```
import smtplib
from email.mime.text import MIMEText

def send_email(to, subject, body):
    msg = MIMEText(body, "html")
    msg["Subject"] = subject
    msg["From"] = "acadeval@gmail.com"
    msg["To"] = to
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
        server.login("acadeval@gmail.com", APP_PASSWORD)
        server.send_message(msg)
```


19 Module 15 — Web Platform (Frontend + Backend)

What it is: the deployed, demoable product that ties every module together — React.js frontend, FastAPI backend.

API surface

Endpoint	Method	Does
/auth/login	POST	issues a JWT (Module 12)
/submit	POST	accepts a new proposal file, enqueues the Celery chain (Module 11)
/status/{job_id}	GET	polls pipeline progress
/report/{project_id}	GET	returns the full explainable report (novelty + citation + writing quality)
/faculty-review	POST	stores a faculty rating (Module 10)
/admin/taxonomy	GET/POST	manage AcadEval_DomainTaxonomy and FeatureKnowledgeBase entries

Step-by-step implementation

- 1 Frontend: React.js — login screen, submission form, a results dashboard rendering the report (novelty breakdown, citation quality, writing quality, trend context), and a faculty review screen.
- 2 Backend: FastAPI implementing the endpoints above, each a thin wrapper that either runs synchronously (auth, status polling) or enqueues a Celery task (submission processing).
- 3 Keep the frontend/backend contract documented with FastAPI's automatic OpenAPI schema (available at /docs) so the React team can build against it without guessing.

20 Module 16 — Deployment and DevOps

Step-by-step implementation

- 1 Write a Dockerfile for the FastAPI backend (including spaCy/BERT model downloads at build time to avoid slow cold starts) and a separate one for the Celery worker.
- 2 Add a docker-compose.yml for local development: backend, worker, PostgreSQL, Redis, and a GROBID container.
- 3 Deploy the backend and Celery worker to Render (or a college server) as two separate services sharing the same PostgreSQL and Redis instances.
- 4 Deploy the React frontend to Vercel, pointing its API base URL at the Render backend.
- 5 Use GitHub Actions (Git and GitHub) for CI: run tests and linting on every pull request, and auto-deploy main to Render/Vercel on merge.

```
# docker-compose.yml (excerpt)
services:
  backend:
    build: ./backend
    ports: ["8000:8000"]
    depends_on: [postgres, redis]
  worker:
    build: ./backend
    command: celery -A app.celery_app worker --loglevel=info
    depends_on: [postgres, redis]
  postgres:
    image: postgres:16
  redis:
    image: redis:7
  grobid:
    image: grobid/grobid:0.8.0
```

21 Deferred: iThenticate Integration

iThenticate (document-level plagiarism/originality checking) is intentionally **left out of this implementation plan**. It solves a different, narrower problem — text similarity against a publisher database — than the novelty engine in Module 5, and integrating it cleanly (API access, licensing, and deciding how its score sits alongside the Novelty Index in the report) deserves its own focused pass once the core platform above is working end to end.

Placeholder for later: when ready, iThenticate would plug in as an additional Celery task alongside Module 6 (Citation Analysis) — same pattern, same report section — without requiring changes to any other module.

22 Build Order and Milestones

Phase	Build	Depends On
1	Datasets: Corpus, DomainTaxonomy, FeatureKnowledgeBase (Section 2)	None
2	Module 12 (Auth) + Module 15 skeleton (empty React app + FastAPI hello-world) + Module 16 (docker-compose for local dev)	None — infra-only
3	Module 1 (Parsing) + Module 11 (Celery/Redis wiring)	Phase 2
4	Module 2 (Domain Classification) + Module 3 (Entity Extraction)	Phase 1 datasets, Phase 3
5	Module 4 (Graph Construction)	Phase 4
6	Module 5 (Novelty Engine v1 — composite index)	Phase 5
7	Module 6 (Citation Analysis) + Module 7 (Writing Quality) + Module 8 (Trend Scoring)	Phase 3
8	Module 9 (Explainable AI) + Module 10 (Faculty Review) + Module 14 (Notifications)	Phases 6-7
9	Collect faculty ratings; train Novelty Engine v2 (Module 5 regression + SHAP)	Phase 8, real usage data
10	Module 13 (External Integrations: GitHub, Moodle) + production deploy (Module 16: Render/Vercel)	Phase 9
11 (later)	iThenticate integration (Section 21)	Phase 10